

AI HR Resume and LinkedIn Shortlisting Agent

Intelligent Candidate Screening, Ranking, Analytics and Reporting Platform

Candidate Name	Shagun Talwar
SAP ID	500119092
College Email ID	shagun.119092@stu.upes.ac.in
Personal Email ID	shgntlwr@gmail.com
Git Repository	https://github.com/shaguntalwar17/HR-Resume-and-LinkedIn-Shortlisting-Agent
Project Domain	Artificial Intelligence, HR Technology, Full-Stack Web Engineering

Certification

This is to certify that **Shagun Talwar** (SAP ID: **500119092**) from UPES, Dehradun has prepared the project report entitled **AI HR Resume and LinkedIn Shortlisting Agent**. The work documents a full-stack AI-assisted recruitment shortlisting platform that supports resume upload, LinkedIn profile ingestion, candidate evaluation, ranking, reporting, analytics, authentication, authorization and UML-based technical documentation. The report is prepared for final-year engineering submission and academic assessment.

Declaration

I, **Shagun Talwar**, hereby declare that this project report titled **AI HR Resume and LinkedIn Shortlisting Agent** represents my project work and technical documentation. The system described in this report is based on the repository implementation and the accompanying UML documentation assets. All external tools, libraries and references are acknowledged in the bibliography.

Acknowledgement

I express my sincere gratitude to my institution, faculty mentors, peers and technical communities whose resources and guidance supported this project. I also acknowledge the open-source ecosystem behind Next.js, React, Prisma, PlantUML, PDF processing libraries and AI-assisted engineering workflows, which enabled the implementation of a practical HR shortlisting solution.

Abstract

The **AI HR Resume and LinkedIn Shortlisting Agent** is a modern recruitment technology platform designed to reduce manual screening effort and improve the consistency of candidate evaluation. The repository describes a Next.js full-stack application, backed by Prisma-managed relational persistence, role-aware authentication, resume/LinkedIn profile ingestion, AI-inspired scoring logic, analytics dashboards, PDF reporting and a complete UML/OOAD documentation set. Recruiters create job requisitions, upload candidate resumes and LinkedIn JSON/profile data, trigger evaluation, review score breakdowns, apply human overrides and generate shortlist reports. Hiring managers review assigned candidates, while administrators manage organizational settings and role-based access.

The solution emphasizes explainable screening rather than opaque automation. Candidate evaluation is represented through structured dimensions such as skills match, experience relevance, education alignment, domain suitability and overall recommendation. The architecture also includes audit logging, guard-based authorization, safe environment configuration, file parsing controls and secure LLM interaction practices. UML generation is automated using a Node.js script that creates PlantUML source files, SVG renders and a master markdown file for project documentation.

Contents

1	Introduction	1
2	Problem Statement	1
3	Objectives	1
4	Existing System Analysis	1
5	Proposed System	1
6	Scope of the Project	2
7	Technology Stack	2
8	System Architecture	3
9	Documentation of LLM Model Chosen	3
9.1	Model Selection Rationale	3
9.2	Prompt Engineering Strategy	3
9.3	Evaluation Strategy	4
10	Agent Framework Used	4
11	Security and Risk Mitigation	4
12	Database Design	5
13	Frontend Design	6
14	Backend Design	6
15	Authentication and Authorization	6
16	Resume Upload and Parsing Workflow	6
17	LinkedIn Data Ingestion Flow	7
18	AI Candidate Evaluation Logic	7
19	Candidate Scoring and Ranking System	7

20 Analytics and Reporting Module	7
21 API Design	7
22 Workflow Explanation	8
23 Algorithms and Internal Logic	8
24 Screenshots with Explanation	8
25 Testing Strategy	15
26 Test Cases	16
27 Results and Outputs	16
28 Advantages of the System	16
29 Limitations	16
30 Future Enhancements	17
31 Challenges Faced	17
32 Conclusion	17
33 References / Bibliography	17
A Appendix: UML Automation Implementation	17
B Appendix: Complete PlantUML Diagram Inventory	18
C Appendix: Project Metadata	18

List of Figures

1	High-level architecture of the shortlisting platform	3
2	Landing page of the HireWise AI production HR shortlisting platform	9
3	Sign-in interface and demo credentials panel	9
4	Recruiter dashboard with status cards, source distribution and hiring alerts	10
5	Job requisition management screen with role requirements and AI match configuration	10
6	Candidate upload and candidate discovery screen	11

7	Evaluations module with filters and scored candidate rows	11
8	Shortlist report generation panel	12
9	Reports module with resume analytics, AI evaluation intelligence and shortlist exports	12
10	Generated PDF report preview in the browser	13
11	Analytics dashboard with timeline, average score and funnel insights	13
12	Audit log screen for traceability and compliance monitoring	14
13	Responsible AI governance page	14
14	Settings page for scoring and AI configuration	15
15	Settings page for organization profile and user role configuration	15

List of Tables

1	Technology Stack	2
2	Security Risks and Mitigations	4
3	Database Entity Summary	6
4	Representative Evaluation Dimensions	7
5	API Design Overview	7
6	Representative Test Cases	16

1 Introduction

Recruitment teams frequently evaluate large volumes of resumes, LinkedIn profiles and job requirements under time pressure. Manual review is slow, subjective and difficult to audit. This project implements an AI-assisted HR shortlisting platform, referred to in the UML documentation as **HireWise AI**, that helps recruiters structure candidate evaluation and create explainable shortlists.

The application combines a web interface, backend APIs, Prisma data models, scoring services, file parsers, authentication guards, audit logs, reporting utilities and UML documentation. It is intended as both a functional HR screening tool and a technically demonstrable final-year project.

2 Problem Statement

Traditional candidate screening suffers from five major issues: high recruiter workload, inconsistent interpretation of job descriptions, fragmented resume and LinkedIn data, limited explainability in shortlisting decisions and weak documentation of screening workflows. The project addresses these issues by providing a centralized platform where job requisitions, candidate profiles, AI evaluation outputs, recruiter reviews and shortlist reports are managed in a traceable workflow.

3 Objectives

- Build a full-stack recruitment shortlisting platform for resumes and LinkedIn-derived candidate data.
- Provide structured job requisition management with required and preferred skill fields.
- Parse uploaded resume documents and normalize LinkedIn/profile JSON into candidate records.
- Evaluate candidates against job requirements using a dimension-based scoring engine.
- Generate explainable score breakdowns, recommendations and shortlist statuses.
- Support recruiter review, human override and audit logging.
- Provide analytics dashboards and downloadable reporting artifacts.
- Document the system using complete UML/OOAD diagrams and automated PlantUML generation.

4 Existing System Analysis

Existing HR screening practices generally rely on manual resume reading, spreadsheet tracking, standalone applicant tracking systems or keyword filters. These approaches can miss context, do not always explain ranking decisions and often separate recruiter notes from analytics. Many keyword filters also over-rank candidates who repeat terms without proving experience. The proposed system improves on this by combining structured job data, normalized candidate profiles, multi-dimensional scoring and recruiter-controlled final decision-making.

5 Proposed System

The proposed system is a role-based web application with the following workflow: an administrator configures organization settings, a recruiter creates a job requisition, uploads resumes and LinkedIn

JSON data, triggers candidate evaluation, views ranked candidates, records decisions or overrides and generates reports. Hiring managers review assigned candidates through a controlled workflow. The backend persists candidates, jobs, evaluations, score breakdowns, reviews and audit logs through Prisma.

Key Implementation Characteristics

- **Frontend:** Next.js App Router and React components for dashboards, forms and review screens.
- **Backend:** API route handlers for candidate, job, evaluation, authentication and reporting workflows.
- **Persistence:** Prisma schema with entities including Organization, User, JobRequisition, Candidate, ApplicationEvaluation, ScoreBreakdown, RecruiterReview and AuditLog.
- **AI/Scoring:** Platform evaluator service that compares candidate profiles to job requirements and produces structured recommendations.
- **Documentation:** Automated UML generation using PlantUML encoding and remote SVG rendering.

6 Scope of the Project

The project scope includes organization-level recruitment management, job requisition creation, candidate upload, resume text extraction, LinkedIn profile ingestion, AI-assisted scoring, recruiter review, hiring manager assignment, analytics, report generation and technical documentation. It does not replace recruiter judgment; instead, it provides decision support with human override and auditability.

7 Technology Stack

Table 1: Technology Stack

Layer	Technologies / Repository Indicators
Frontend	Next.js App Router, React UI components, dashboard pages, role-based routes
State/UI	Zustand state store indicated in component diagram, reusable component library
Backend	Next.js API route handlers, server-side guards and proxy middleware
Database	Prisma ORM with SQLite/PostgreSQL-compatible persistence
File Processing	Resume parsers using pdf2json-style extraction and mammoth-style document handling
Reporting	PDF report generation using pdf-lib-style server utilities
Authentication	Session token/JWT or cookie-based flow validated by proxy middleware and role guards
Documentation	PlantUML, generated .puml files, SVG diagrams and all-diagrams-code.md
Deployment	Docker/VPS/cloud hosting, persistent volume, public uploads and database file path

8 System Architecture

The system follows a modular full-stack architecture. The browser renders React pages and submits requests to Next.js API routes. Middleware validates sessions before protected routes are executed. API handlers call services in the library layer: authentication guards, parsers, scoring/evaluator logic and report generation. Prisma acts as the database access layer. Persistent storage holds structured records and uploaded documents.

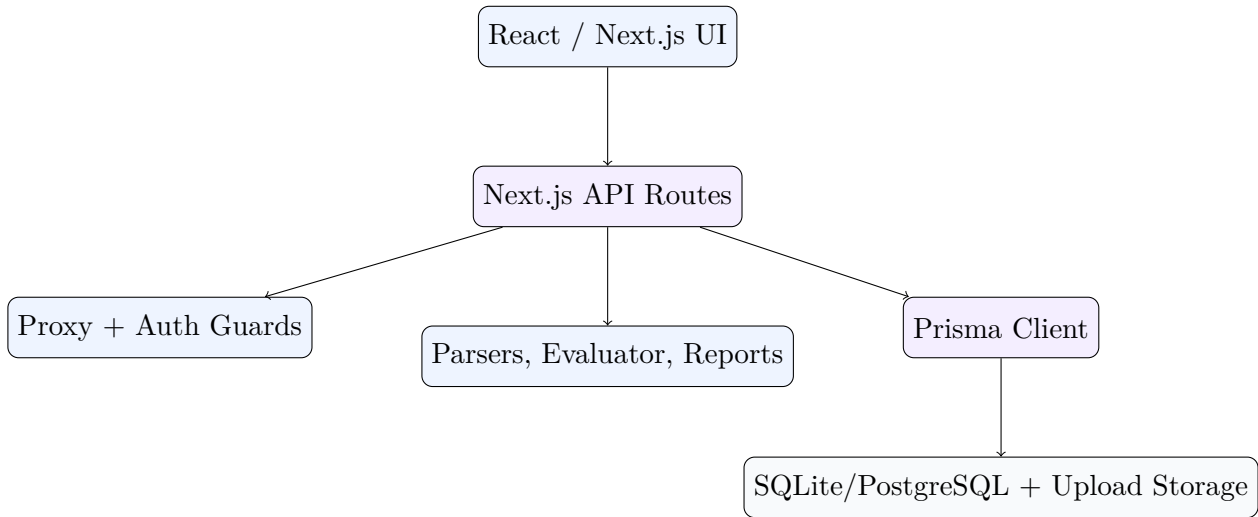


Figure 1: High-level architecture of the shortlisting platform

9 Documentation of LLM Model Chosen

The repository documentation positions the evaluation module as an AI candidate evaluator. The implementation described by the UML uses a **Platform Evaluator / ScoringEngine** that can be used directly as deterministic scoring logic and can also be extended with an LLM for explanation generation. For HR screening, the recommended LLM profile is a structured-output model capable of following system prompts, returning JSON, explaining reasoning compactly and avoiding direct demographic inference.

9.1 Model Selection Rationale

The model should support long-context resume/JD comparison, robust JSON output, instruction hierarchy, safety controls, low hallucination tendency and strong summarization. Such a model is suitable because HR screening requires matching skills and experience while producing auditable justifications rather than untraceable decisions.

9.2 Prompt Engineering Strategy

- Use a strict system prompt that defines the model as an assistant evaluator, not the final decision-maker.
- Provide normalized job requirements and candidate profile fields instead of raw uncontrolled text wherever possible.
- Require JSON output with dimensions, scores, evidence snippets and recommendation.

- Exclude protected attributes and instruct the model to ignore irrelevant personal information.
- Apply server-side validation to every model response before persisting it.

9.3 Evaluation Strategy

Evaluation should compare AI outputs with recruiter decisions using precision of shortlist recommendations, consistency across similar candidates, stability of score breakdowns and quality of justifications. Recruiter override data can be used to identify weak scoring dimensions and refine prompts or deterministic weights.

10 Agent Framework Used

The project uses an agentic workflow rather than a single isolated function. The documented agent flow contains ingestion, normalization, scoring, explanation, persistence, review and reporting stages. The main agent role is the **AI shortlisting agent**; it receives job requisition data and candidate data, invokes parsing/scoring tools, returns structured evaluation results and records outputs through API handlers.

Agent Orchestration Flow

1. Recruiter uploads resumes and LinkedIn JSON/profile data.
2. API route validates role and input payload.
3. Parser extracts text and maps candidate attributes.
4. Evaluator compares normalized candidate data with job requisition requirements.
5. Score breakdowns and recommendations are persisted.
6. Recruiter reviews, overrides if needed and assigns candidates to hiring managers.
7. Reporting module generates shortlist output and analytics summaries.

11 Security and Risk Mitigation

Table 2: Security Risks and Mitigations

Risk	Mitigation and Implementation Approach
Prompt Injection	Treat resume and LinkedIn content as untrusted data. Use system prompts that forbid instruction following from candidate documents, isolate candidate text in delimited fields and validate structured model output server-side.
Data Privacy	Store only recruitment-relevant data, restrict access by organization and role, avoid exposing parsed resume text to unauthorized users and define retention policies.
Credential Handling	Use environment variables for database URLs, auth secrets and model keys. Never hard-code secrets in source control.
Authentication	Session/JWT/cookie validation occurs through proxy middleware before protected routes are served. Invalid sessions redirect to login.

Authorization	Role guard functions such as <code>requireRole</code> enforce ADMIN, RECRUITER, HIRING MANAGER and VIEWER boundaries before API data access.
Session Management	Validate token integrity, expiration and backing user existence. Use secure, HTTP-only cookies in production deployments.
File Upload Security	Restrict accepted file types, apply size limits, store files in controlled upload paths and parse documents server-side using safe libraries.
Resume Data Protection	Avoid public exposure of private uploads, protect document URLs and log only metadata rather than sensitive resume contents.
Input Validation	Validate job requirements, candidate JSON, numeric scores, enums and IDs before database writes.
SQL Injection	Prisma parameterized query generation avoids direct string-concatenated SQL for normal CRUD operations.
API Security	Require authentication on protected endpoints, return least-privilege data and handle errors without leaking stack traces.
Sensitive Information	Mask secrets and private candidate details in logs, reports and exceptions.
Role-Based Access Control	Admins manage organization settings, recruiters manage screening, hiring managers review assigned candidates and viewers receive read-only access.
Audit Logging	Record sensitive actions such as evaluation runs, overrides, assignments and status changes.
Safe LLM Interaction	Send minimum necessary fields, remove protected attributes, constrain outputs and keep humans in the final decision loop.
Secure Parsing	Prefer maintained parsers, handle parse failures gracefully and never execute uploaded file content.
Data Isolation	Organization relationships in the data model support tenant-level isolation across users, candidates, jobs and logs.
Error Handling	Use structured error responses for clients and private server logs for diagnostics.

12 Database Design

The data model centers on organization-scoped recruitment objects. The key tables/classes are summarized below.

Table 3: Database Entity Summary

Entity	Purpose
Organization	Tenant boundary with name and industry information.
User	Authenticated platform user with role and organization membership.
JobRequisition	Recruiter-created job opening with title, required skills and status.
Candidate	Uploaded or ingested applicant profile with parsed resume text, source and skills.
ApplicationEvaluation	Candidate-versus-job evaluation with score, status and recommendation.
ScoreBreakdown	Dimension-level score and justification linked to an evaluation.
RecruiterReview	Human decision, notes and override reason.
AuditLog	Trace of significant actions such as evaluations and overrides.

13 Frontend Design

The frontend is organized around recruiter productivity and evaluation explainability. Pages include dashboards, job management, candidate upload, evaluation review, analytics and report access. UI components are expected to show status badges, score cards, candidate tables, filters, charts and decision forms. The design supports different roles by hiding or disabling unauthorized actions.

14 Backend Design

The backend is implemented through Next.js server routes and library services. API handlers validate requests, call Prisma for persistence, invoke parsers/evaluators/report generators and return JSON or generated files. The library layer separates authentication, database access, parsing, scoring and reporting concerns, improving maintainability.

15 Authentication and Authorization

Authentication is represented by a middleware/proxy flow that verifies session tokens or cookies before protected pages and API routes are served. Authorization is enforced by role guards, especially `requireRole`. Admins control organization-level settings, recruiters manage jobs and evaluations, hiring managers review assigned candidates and viewers consume read-only analytics or candidate details.

16 Resume Upload and Parsing Workflow

The resume upload workflow accepts candidate documents, validates file metadata, stores the uploaded file, extracts text using parser utilities and maps extracted content into candidate fields such as name, skills, experience and education. The parsed text is retained for evaluation while structured attributes support scoring.

17 LinkedIn Data Ingestion Flow

LinkedIn ingestion is represented as JSON/profile data upload. The system maps profile headline, summary, experience, education, certifications, skills and project details into the Candidate schema. LinkedIn data complements resume text by improving skill and experience coverage.

18 AI Candidate Evaluation Logic

The evaluator compares each candidate against the job requisition. It extracts required and preferred skills, checks overlap with candidate skills, evaluates experience years, considers education/domain alignment and generates a recommendation. The output is not treated as a final hiring decision; it is a ranked recommendation reviewed by recruiters.

19 Candidate Scoring and Ranking System

Table 4: Representative Evaluation Dimensions

Dimension	Weight Intent	Explanation
Skills Match	High	Required and preferred skill overlap with candidate resume/LinkedIn skills.
Experience Relevance	High	Years of experience and similarity of previous responsibilities to the job.
Education / Certification	Medium	Degree, certifications and training relevant to the role.
Domain Fit	Medium	Prior industry or role context aligned with the requisition.
Communication / Profile Quality	Low-Medium	Completeness and clarity of candidate profile evidence.

Candidates are ranked by overall score and recommendation labels such as **STRONG_SHORTLIST**, hold or reject-style statuses. Recruiters can override recommendations with notes and audit trail entries.

20 Analytics and Reporting Module

The analytics module presents hiring pipeline visibility: candidate counts, evaluation status distribution, score ranges, shortlisted candidates, review decisions and recruiter activity. The reporting module generates PDF-style shortlist artifacts containing job details, ranked candidates, score breakdowns and recruiter notes.

21 API Design

Table 5: API Design Overview

Endpoint Category	Method	Purpose
-------------------	--------	---------

Authentication	POST/GET	Login, session validation and logout flows.
Jobs	GET/POST/PATCH	Create, list and update job requisitions.
Candidates	GET/POST	Upload resumes, ingest LinkedIn data and list candidates.
Evaluation	POST	Run candidate-versus-job evaluation and persist score breakdowns.
Reviews	POST/PATCH	Record recruiter decisions, notes and overrides.
Analytics	GET	Return dashboard metrics and visual summaries.
Reports	GET/POST	Generate or download shortlist reports.

22 Workflow Explanation

The operational workflow begins with authenticated access. A recruiter creates a requisition, uploads candidate data, triggers evaluation and reviews ranked results. The system creates score breakdowns and audit logs. Strong candidates can be assigned to a hiring manager. The hiring manager approves, rejects or progresses candidates. Reports summarize the pipeline for review and submission.

23 Algorithms and Internal Logic

Listing 1: Simplified candidate evaluation logic

```
function evaluateCandidate(candidate, job) {
  const required = normalize(job.requiredSkills);
  const preferred = normalize(job.preferredSkills || []);
  const skills = normalize(candidate.skills || []);
  const requiredMatch = overlap(required, skills) / Math.max(required.length, 1);
  const preferredMatch = overlap(preferred, skills) / Math.max(preferred.length, 1);
  const skillsScore = 70 * requiredMatch + 30 * preferredMatch;
  const experienceScore = scoreExperience(candidate.experienceYears, job.minExperience);
  const educationScore = scoreEducation(candidate.education, job.educationCriteria);
  const domainScore = scoreDomain(candidate.profileText, job.description);
  const overallScore = weightedAverage([
    [skillsScore, 0.40], [experienceScore, 0.30],
    [educationScore, 0.15], [domainScore, 0.15]
  ]);
  return { overallScore, recommendation: classify(overallScore) };
}
```

24 Screenshots with Explanation

This section presents the actual user-interface screenshots supplied for the project. The screenshots document the major modules of HireWise AI, including the landing page, authentication, dashboard, jobs, candidate ingestion, evaluations, reports, analytics, audit logs, Responsible AI and settings.

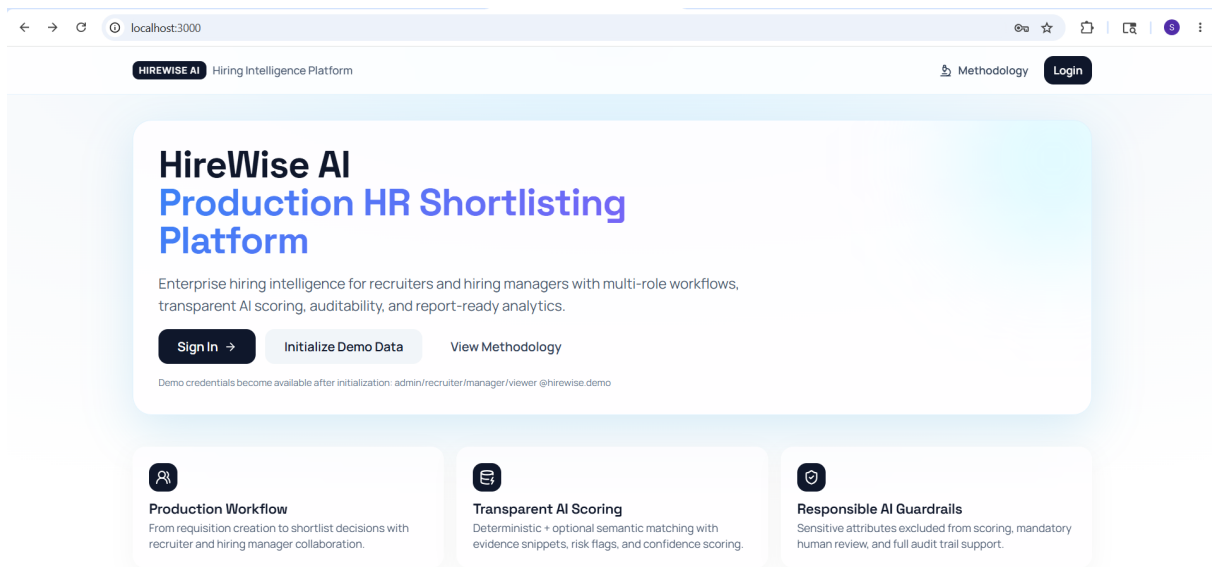


Figure 2: Landing page of the HireWise AI production HR shortlisting platform

The landing page communicates the core value proposition of the platform: resume/profile analysis, transparent AI ranking and responsible AI shortlisting. It provides entry points for authentication and product exploration.

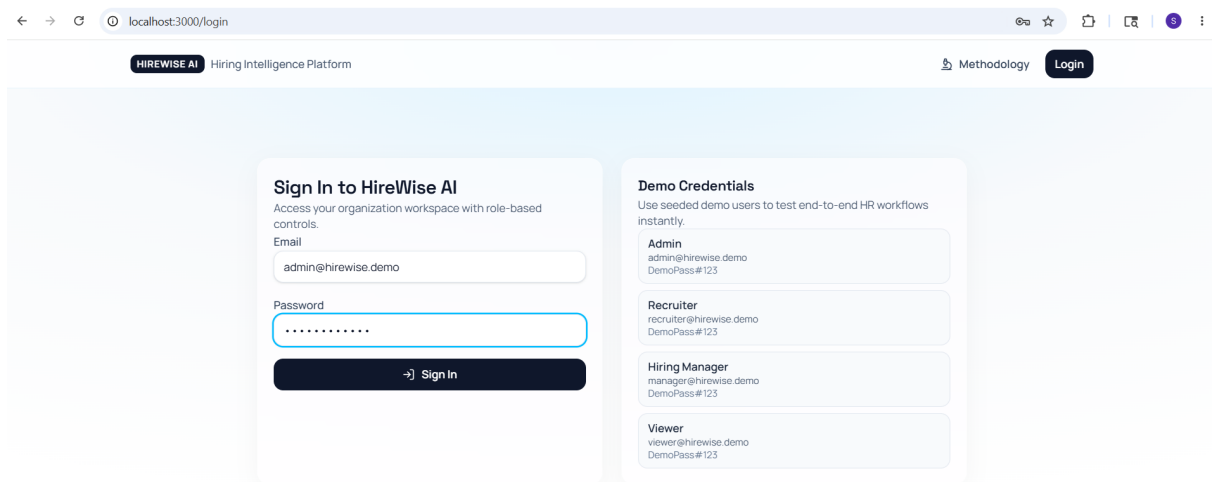


Figure 3: Sign-in interface and demo credentials panel

The authentication screen enables users to enter credentials and also documents demo access roles. This supports evaluation, demonstration and viva walkthroughs without needing production credentials.

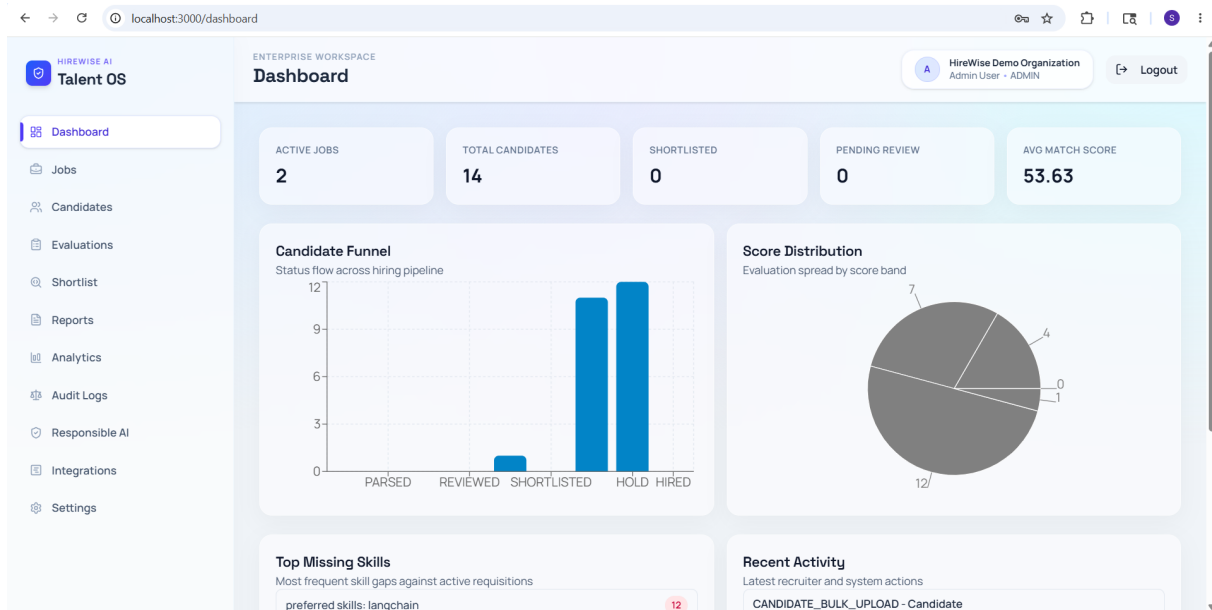


Figure 4: Recruiter dashboard with status cards, source distribution and hiring alerts

The dashboard summarizes recruitment activity using key performance indicators, candidate pipeline charts, source distribution and hiring alerts. It acts as the command center for recruiters and administrators.

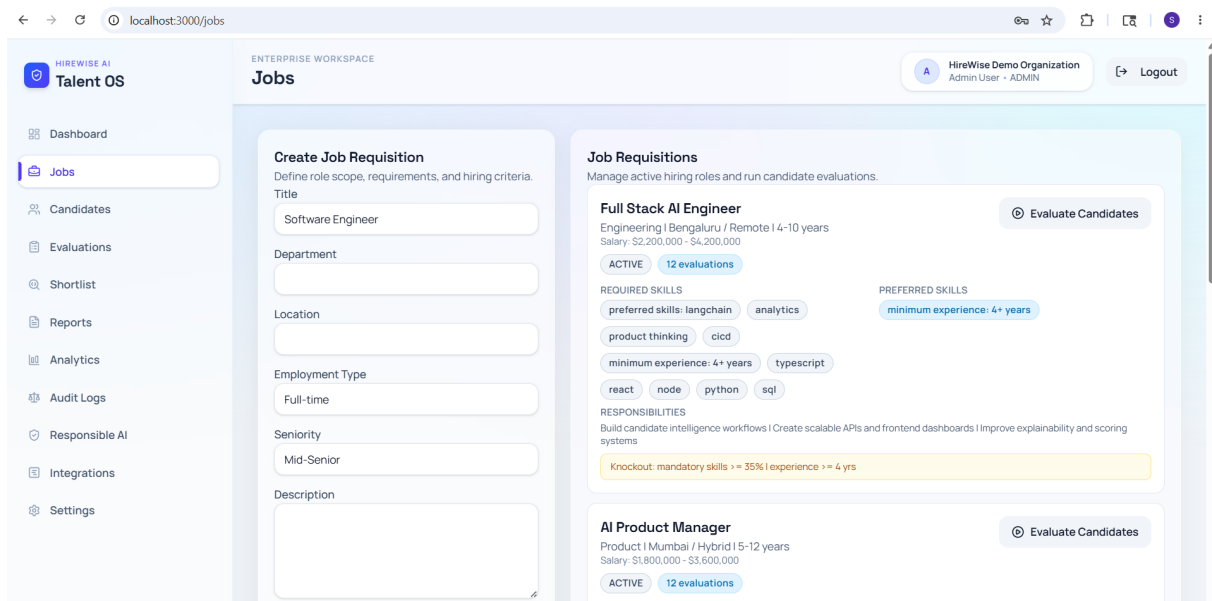


Figure 5: Job requisition management screen with role requirements and AI match configuration

The jobs module allows recruiters to define job descriptions, required skills, preferred skills, experience expectations and hiring manager assignment. These structured fields become the input for candidate scoring.

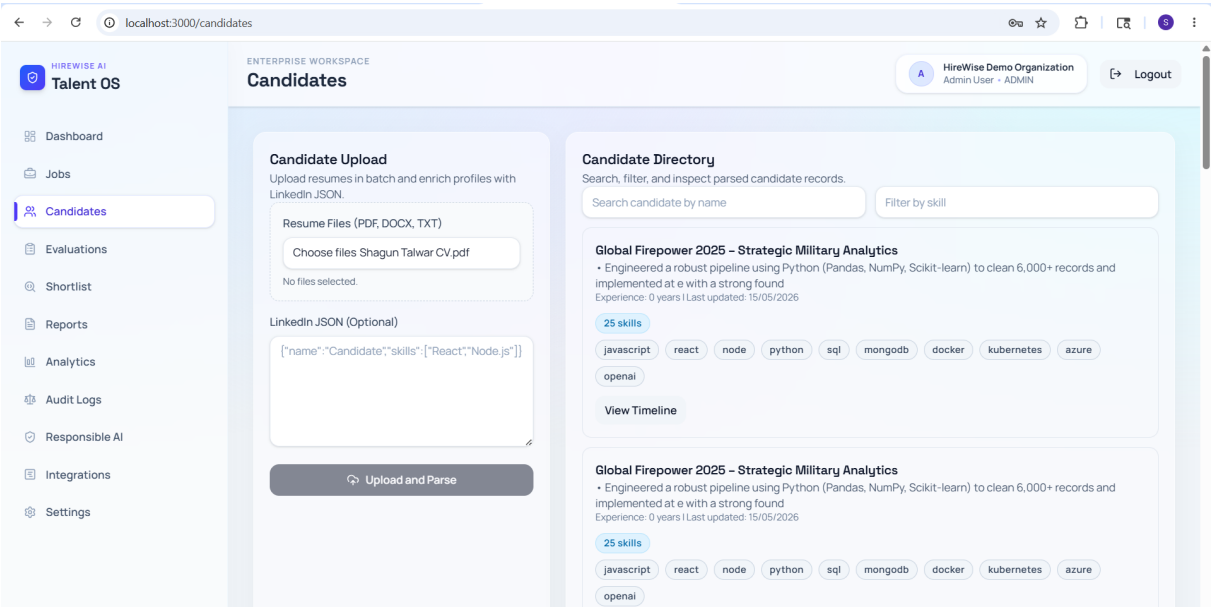


Figure 6: Candidate upload and candidate discovery screen

The candidate screen supports resume upload and candidate profile management. It shows parsed candidate cards and allows recruiters to review candidates after ingestion.

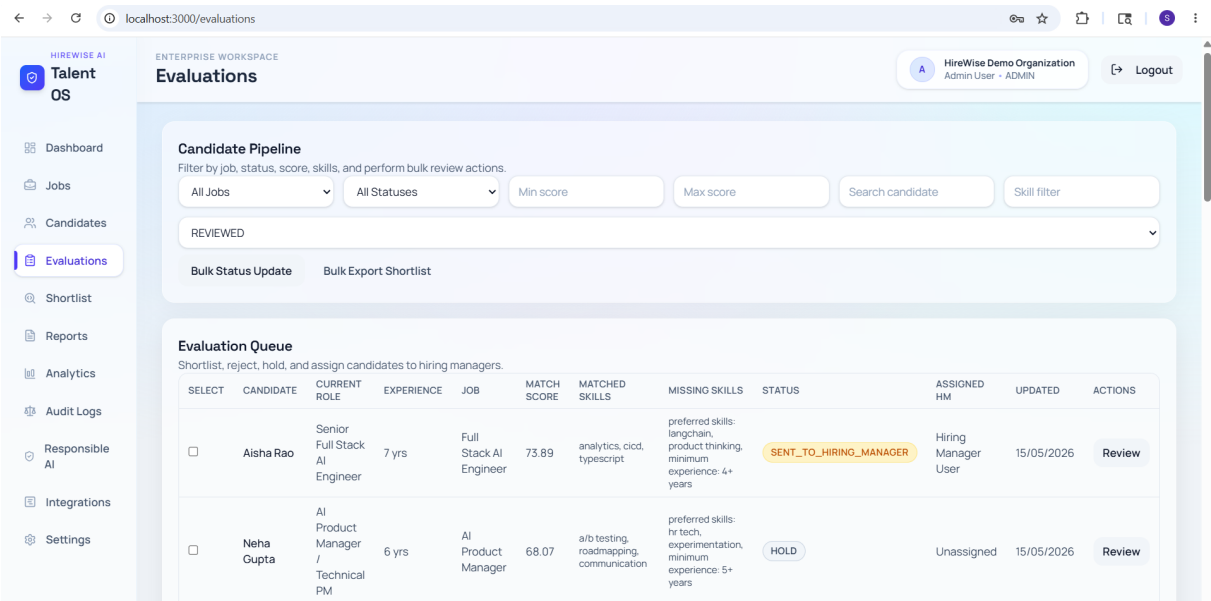


Figure 7: Evaluations module with filters and scored candidate rows

The evaluations module provides a tabular shortlist view with candidate scores, status, recommendation and review information. Filters help recruiters focus on specific job roles, recommendation types or review states.

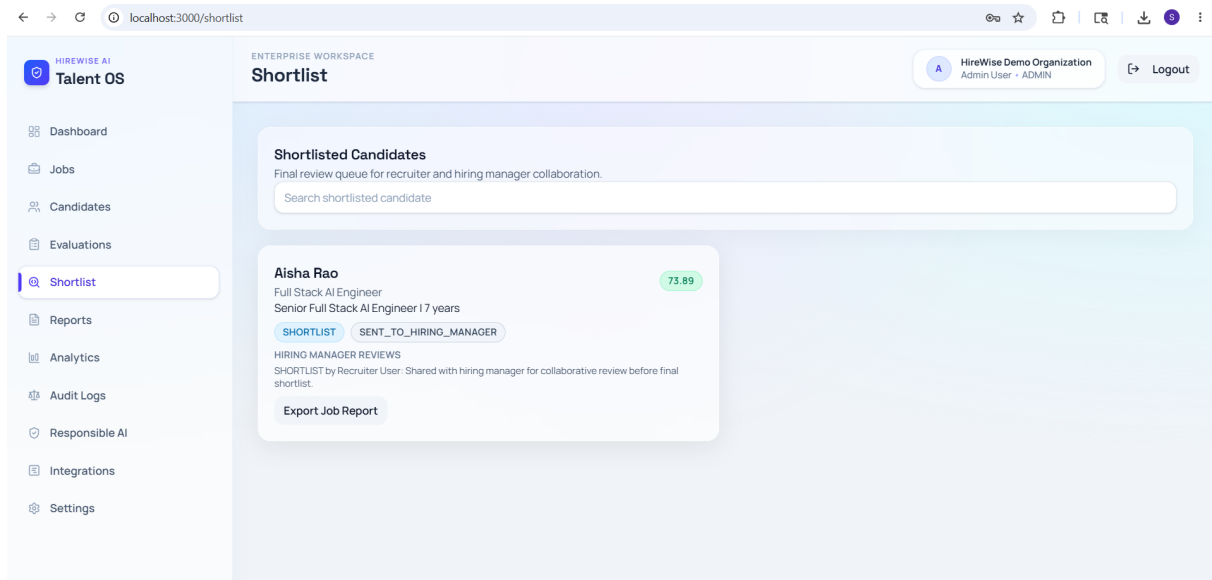


Figure 8: Shortlist report generation panel

The shortlist screen shows report generation output for a selected role. It presents generated report metadata and provides an export/download path for recruiter submission.

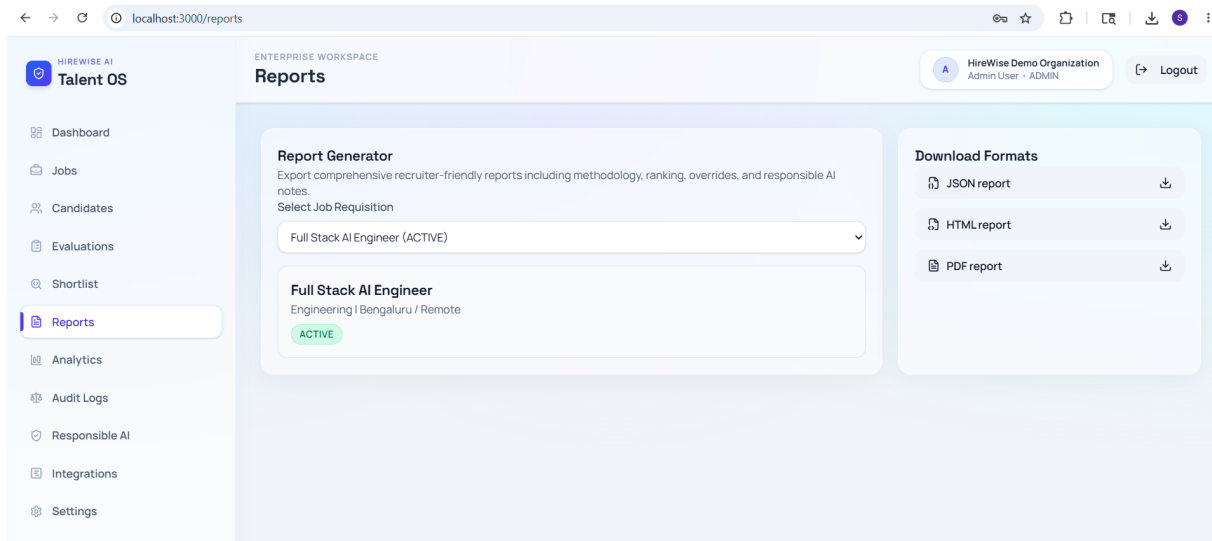


Figure 9: Reports module with resume analytics, AI evaluation intelligence and shortlist exports

The reports page consolidates generated documents, resume analytics and AI evaluation summaries. This module supports offline review and formal HR documentation.

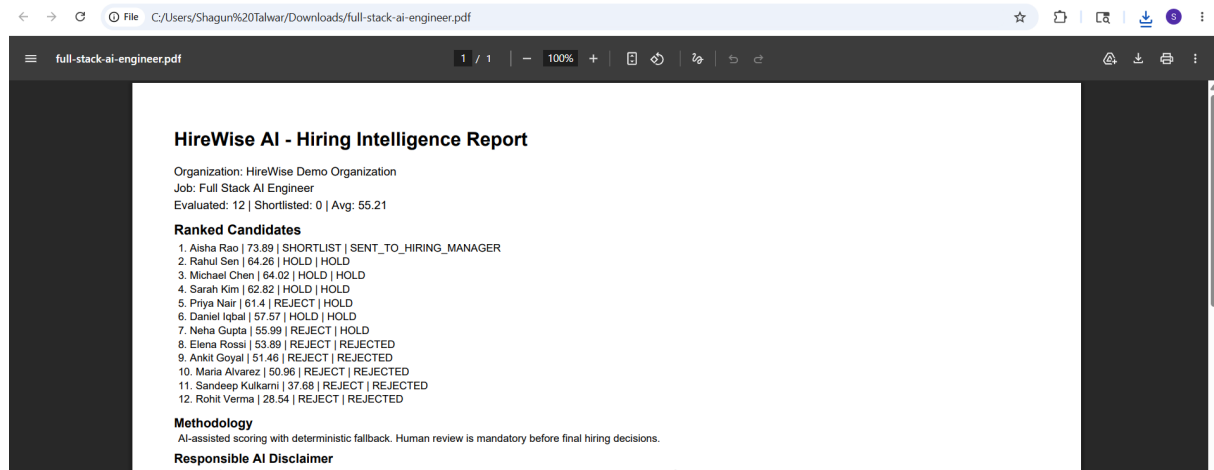


Figure 10: Generated PDF report preview in the browser

The PDF preview demonstrates the final reporting output generated by the system. It includes candidate-ranking information and structured shortlist content suitable for sharing with stakeholders.

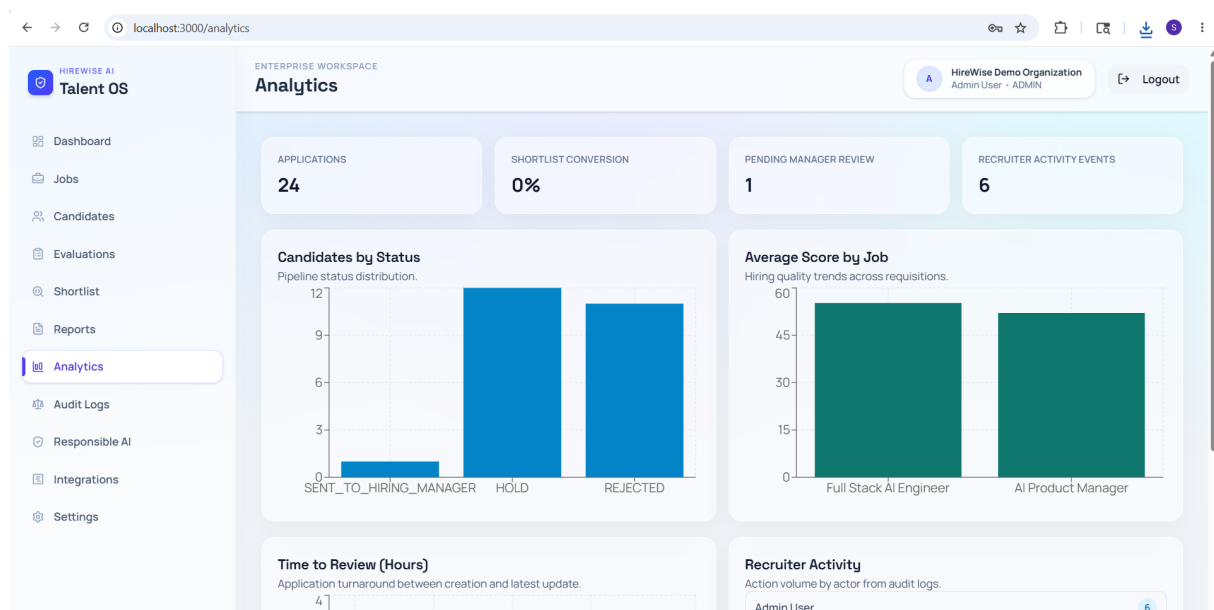
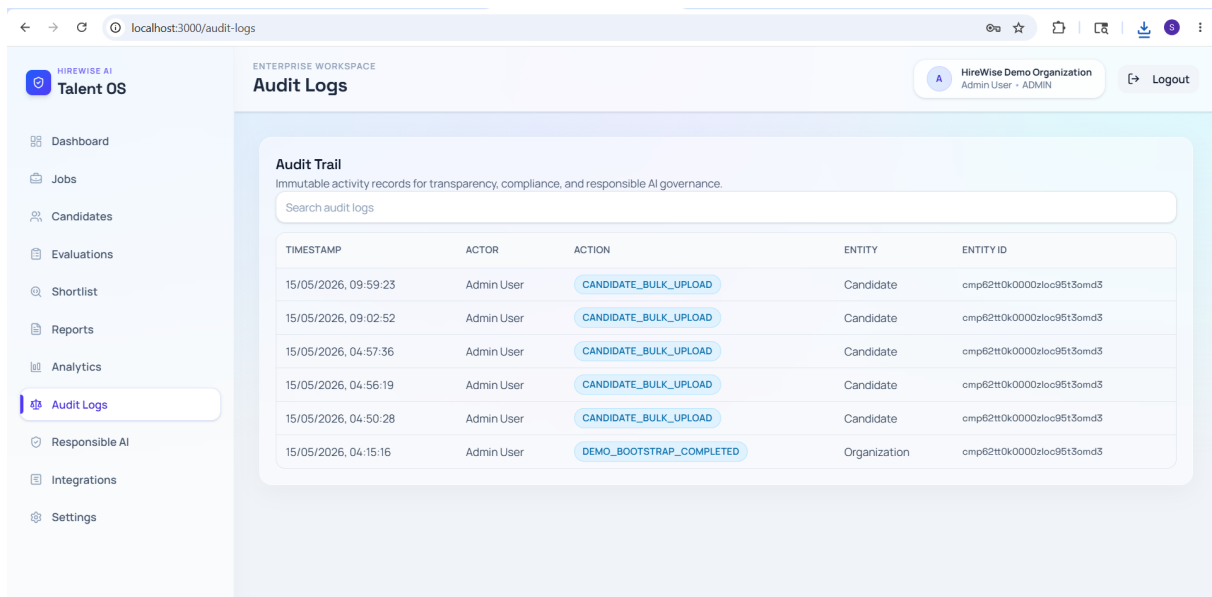


Figure 11: Analytics dashboard with timeline, average score and funnel insights

The analytics screen visualizes hiring metrics such as total jobs, candidates, evaluations, average score, timeline trends and source/category distributions. These charts help teams measure hiring pipeline performance.



HIREWISE AI Talent OS

ENTERPRISE WORKSPACE

Audit Logs

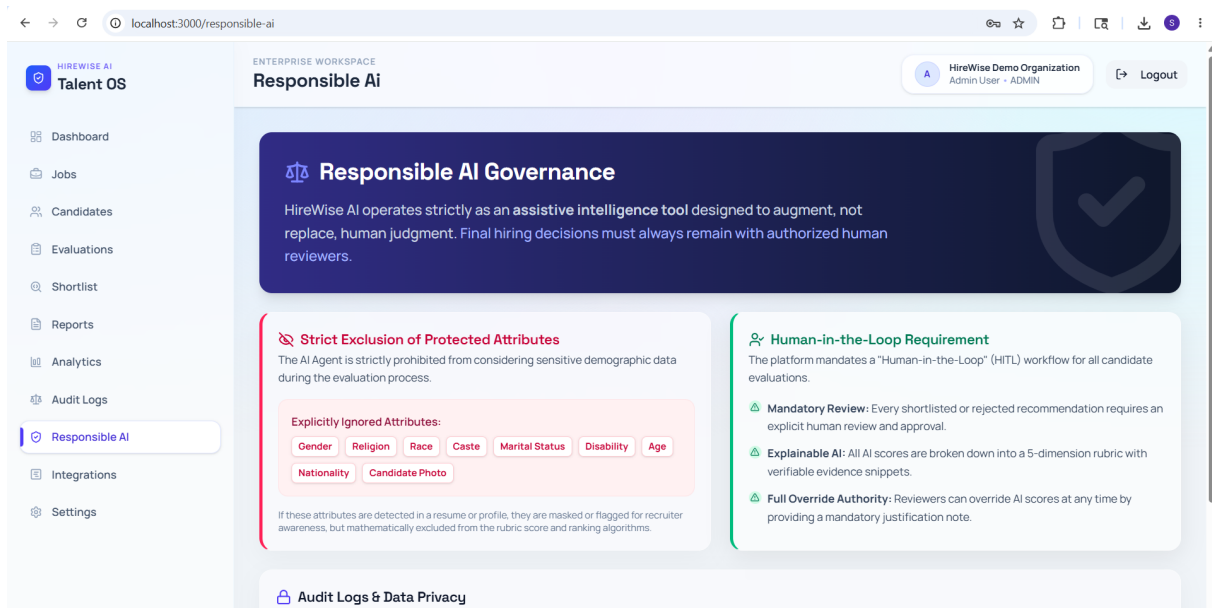
Immutable activity records for transparency, compliance, and responsible AI governance.

Search audit logs

TIMESTAMP	ACTOR	ACTION	ENTITY	ENTITY ID
15/05/2026, 09:59:23	Admin User	CANDIDATE_BULK_UPLOAD	Candidate	cmp62tt0k0000zloc9513omd3
15/05/2026, 09:02:52	Admin User	CANDIDATE_BULK_UPLOAD	Candidate	cmp62tt0k0000zloc9513omd3
15/05/2026, 04:57:36	Admin User	CANDIDATE_BULK_UPLOAD	Candidate	cmp62tt0k0000zloc9513omd3
15/05/2026, 04:56:19	Admin User	CANDIDATE_BULK_UPLOAD	Candidate	cmp62tt0k0000zloc9513omd3
15/05/2026, 04:50:28	Admin User	CANDIDATE_BULK_UPLOAD	Candidate	cmp62tt0k0000zloc9513omd3
15/05/2026, 04:15:16	Admin User	DEMO_BOOTSTRAP_COMPLETED	Organization	cmp62tt0k0000zloc9513omd3

Figure 12: Audit log screen for traceability and compliance monitoring

The audit log records important platform actions such as evaluation runs, uploads, reviews and configuration changes. It improves accountability and supports compliance-oriented review.



HIREWISE AI Talent OS

ENTERPRISE WORKSPACE

Responsible AI

Responsible AI Governance

HireWise AI operates strictly as an assistive intelligence tool designed to augment, not replace, human judgment. Final hiring decisions must always remain with authorized human reviewers.

Strict Exclusion of Protected Attributes

The AI Agent is strictly prohibited from considering sensitive demographic data during the evaluation process.

Explicitly Ignored Attributes:

- Gender
- Religion
- Race
- Caste
- Marital Status
- Disability
- Age
- Nationality
- Candidate Photo

If these attributes are detected in a resume or profile, they are masked or flagged for recruiter awareness, but mathematically excluded from the rubric score and ranking algorithms.

Human-in-the-Loop Requirement

The platform mandates a "Human-in-the-Loop" (HITL) workflow for all candidate evaluations.

- Mandatory Review:** Every shortlisted or rejected recommendation requires an explicit human review and approval.
- Explainable AI:** All AI scores are broken down into a 5-dimension rubric with verifiable evidence snippets.
- Full Override Authority:** Reviewers can override AI scores at any time by providing a mandatory justification note.

Audit Logs & Data Privacy

Figure 13: Responsible AI governance page

The Responsible AI page documents governance principles, fairness controls, transparency commitments and mitigation strategies. It reinforces that AI recommendations are assistive and must remain under human supervision.

The screenshot shows the 'Settings' page for 'HireWise AI Talent OS'. The left sidebar contains navigation links: Dashboard, Jobs, Candidates, Evaluations, Shortlist, Reports, Analytics, Audit Logs, Responsible AI, and Integrations. The 'Settings' link is highlighted. The main content area is titled 'Settings' and 'ENTERPRISE WORKSPACE'. It features a 'Scoring & AI Configuration' section with input fields for SKILLSMATCH (0.3), EXPERIENCERELEVANCE (0.25), EDUCATIONCERTS (0.15), PROJECTPORTFOLIO (0.2), and COMMUNICATIONQUALITY (0.1). Below these are fields for Minimum Score Threshold (70), Mandatory Skill Match % (35), Minimum Experience (Years), and Reject on Missing Mandatory Skills (>=). A checkbox for 'Enable AI enhancement where API key is available' is present, along with a 'Save Configuration' button. The 'Organization Profile' section includes fields for Organization Name (HireWise Demo Organization) and Industry (HR Technology), with a 'Save Organization' button. The 'User Roles' section lists roles: Admin User (admin@hirewise.demo), Recruiter User (recruiter@hirewise.demo), Hiring Manager User (manager@hirewise.demo), and Viewer User (viewer@hirewise.demo), each with a dropdown menu showing the assigned role (ADMIN, RECRUITER, HIRING_MANAGER, VIEWER).

Figure 14: Settings page for scoring and AI configuration

The settings module exposes scoring and configuration controls used by the platform. This supports organization-level tuning of evaluation behavior and operational preferences.

This screenshot shows the 'Settings' page for 'HireWise AI Talent OS', focusing on the 'Organization Profile' and 'User Roles' sections. The left sidebar is the same as in Figure 14. The 'Organization Profile' section has a 'Save Configuration' button at the top, followed by fields for Organization Name (HireWise Demo Organization) and Industry (HR Technology), with a 'Save Organization' button. The 'User Roles' section is titled 'Manage recruiter and hiring manager access levels.' and lists four roles with their email addresses and assigned roles in dropdown menus: Admin User (admin@hirewise.demo, ADMIN), Recruiter User (recruiter@hirewise.demo, RECRUITER), Hiring Manager User (manager@hirewise.demo, HIRING_MANAGER), and Viewer User (viewer@hirewise.demo, VIEWER).

Figure 15: Settings page for organization profile and user role configuration

The organization settings screen manages profile details and role/user configuration. It complements the authentication and authorization design by linking users to organizational roles.

25 Testing Strategy

Testing should cover unit tests for parsing/scoring utilities, integration tests for API route behavior, role-based authorization tests, file upload validation, database persistence tests and UI workflow checks. Security testing should include invalid sessions, unauthorized role access, oversized uploads, malformed JSON and prompt-injection strings in resume text.

26 Test Cases

Table 6: Representative Test Cases

ID	Scenario	Input	Expected Output
TC01	Valid recruiter login	Correct credentials/session	Dashboard access granted.
TC02	Unauthorized viewer action	Viewer attempts evaluation	403 Forbidden.
TC03	Resume upload	Valid PDF/DOCX	Candidate created with parsed text.
TC04	Malformed LinkedIn JSON	Invalid JSON payload	Validation error, no database write.
TC05	Evaluation run	Job and candidates present	Evaluation and score breakdown records created.
TC06	Human override	Recruiter changes recommendation	Review and audit log saved.
TC07	Report generation	Evaluated job requisition	PDF/report artifact generated.
TC08	Prompt injection content	Resume contains malicious instruction	Content treated as data, model/system policy preserved.

27 Results and Outputs

The expected outputs include parsed candidate profiles, ranked candidate lists, score breakdowns with justifications, recruiter review records, hiring manager assignments, analytics summaries and downloadable shortlist reports. The UML generator also outputs `.puml` files, `docs/uml/rendered-images/*.svg`, `all-diagrams-code.md` and `docs/uml/README.md`.

28 Advantages of the System

- Reduces repetitive manual resume screening effort.
- Produces explainable candidate rankings rather than black-box decisions.
- Combines resume and LinkedIn evidence for richer evaluation.
- Supports role-based workflow across recruiters, admins, hiring managers and viewers.
- Maintains auditability through reviews, overrides and logs.
- Provides strong software documentation through UML automation.

29 Limitations

The system depends on quality of uploaded resumes, completeness of LinkedIn data and correctness of job requisition fields. AI-assisted recommendations can still reflect missing data or imperfect weighting.

Final hiring decisions require human review, bias checks and compliance with organizational HR policies.

30 Future Enhancements

Future improvements may include ATS integrations such as Greenhouse webhooks, advanced LLM-based explanation generation, vector search over candidate profiles, interview scheduling, email automation, bias monitoring dashboards, anonymized review mode, richer analytics and multi-tenant SaaS deployment hardening.

31 Challenges Faced

Key challenges include normalizing inconsistent resume formats, mapping LinkedIn profiles to structured candidate schemas, balancing AI automation with human oversight, designing secure file upload handling, implementing role-specific workflows and creating complete UML documentation for a full-stack system.

32 Conclusion

The AI HR Resume and LinkedIn Shortlisting Agent demonstrates a practical, technically strong HR technology platform. It integrates modern web engineering, database design, authentication, AI-assisted scoring, analytics, reporting and formal UML documentation. Its strongest design quality is the balance between automation and accountability: the system accelerates shortlisting while keeping recruiters and hiring managers in control of final decisions.

33 References / Bibliography

1. Next.js Documentation: App Router and API Routes.
2. Prisma Documentation: ORM schema modeling and database client usage.
3. PlantUML Documentation: UML diagram syntax and server rendering.
4. OWASP Application Security Verification Standard and file upload security guidance.
5. PDF and document parsing library documentation for resume ingestion workflows.
6. Repository: <https://github.com/shaguntalwar17/HR-Resume-and-LinkedIn-Shortlisting-Agent>.

A Appendix: UML Automation Implementation

The UML automation script uses Node.js modules `fs`, `path`, `zlib` and `https`. It defines a `diagrams` array containing names, titles, descriptions and PlantUML code for the requested OOAD diagrams. For each diagram, the script writes an individual `.puml` file, appends its code to `all-diagrams-code.md`, encodes the PlantUML source using raw deflate compression and PlantUML's custom 64-character alphabet, fetches SVG output from PlantUML's rendering endpoint and saves it under `docs/uml/rendered-images`. It also creates `docs/uml/README.md` summarizing the documentation set.

Listing 2: Core UML generation workflow from the project documentation

```

const fs = require('fs');
const path = require('path');
const zlib = require('zlib');
const https = require('https');
const UML_DIR = path.join(__dirname, 'docs', 'uml');
const IMG_DIR = path.join(UML_DIR, 'rendered-images');
fs.mkdirSync(IMG_DIR, { recursive: true });
const diagrams = [
  { name: 'use-case', title: 'Use Case Diagram', code: '@startuml\n...\n@enduml' },
  { name: 'class-diagram', title: 'Class Diagram', code: '@startuml\n...\n@enduml' },
  { name: 'activity-diagram', title: 'Activity Diagram', code: '@startuml\n...\n@enduml' },
  { name: 'sequence-auth', title: 'Sequence Diagram - Authentication & Guard Flow', code: '@startuml\n...\n\n@enduml' },
  { name: 'sequence-shortlisting', title: 'Sequence Diagram - AI Shortlisting Flow', code: '@startuml\n...\n\n@enduml' },
  { name: 'state-machine', title: 'State Machine Diagram', code: '@startuml\n...\n@enduml' },
  { name: 'component-diagram', title: 'Component Diagram', code: '@startuml\n...\n@enduml' },
  { name: 'deployment-diagram', title: 'Deployment Diagram', code: '@startuml\n...\n@enduml' },
  { name: 'package-diagram', title: 'Package Diagram', code: '@startuml\n...\n@enduml' },
  { name: 'object-diagram', title: 'Object Diagram', code: '@startuml\n...\n@enduml' },
  { name: 'collaboration-diagram', title: 'Communication / Collaboration Diagram', code: '@startuml\n...\n\n@enduml' },
  { name: 'system-context', title: 'System Context Diagram', code: '@startuml\n...\n@enduml' }
];
function encode64(data) {
  let r = '';
  for (let i = 0; i < data.length; i += 3) {
    r += append3bytes(data[i], data[i + 1] || 0, data[i + 2] || 0);
  }
  return r;
}
function generatePlantUmlUrl(puml) {
  const deflated = zlib.deflateRawSync(Buffer.from(puml, 'utf8'));
  const encoded = encode64(deflated);
  return 'https://www.plantuml.com/plantuml/svg/' + encoded;
}
async function main() {
  let markdownContent = '# HireWise AI - UML & OOAD Diagrams\n\n';
  for (const d of diagrams) {
    fs.writeFileSync(path.join(UML_DIR, d.name + '.puml'), d.code);
    markdownContent += '## ${d.title}\n\n\`'\`plantuml\n${d.code}\n\`'\`'\n\n';
    const url = generatePlantUmlUrl(d.code);
    await fetchSvg(url, path.join(IMG_DIR, d.name + '.svg'));
  }
  fs.writeFileSync(path.join(UML_DIR, 'all-diagrams-code.md'), markdownContent);
}
main();

```

B Appendix: Complete PlantUML Diagram Inventory

The documentation set contains the following diagrams: Use Case Diagram, Class Diagram, Activity Diagram, Authentication Sequence Diagram, AI Shortlisting Sequence Diagram, State Machine Diagram, Component Diagram, Deployment Diagram, Package Diagram, Object Diagram, Collaboration Diagram and System Context Diagram. The full source is maintained in generated .puml files and the master all-diagrams-code.md file.

C Appendix: Project Metadata

- **Project Title:** AI HR Resume and LinkedIn Shortlisting Agent
- **Candidate:** Shagun Talwar
- **SAP ID:** 500119092

- **College Email:** shagun.119092@stu.upes.ac.in
- **Personal Email:** shgntlwr@gmail.com
- **Repository:** <https://github.com/shaguntalwar17/HR-Resume-and-LinkedIn-Shortlisting-Agent>